
Web Application Penetration Test Report — v1.0

Target System: OWASP Juice Shop v19.1.1

Performed by the OpenHack Security Agent

Issued by Jane Doe (jane.doe@openhack.sec)

2026-02-26

Contents

1 Document Control	2
2 Executive Summary	3
3 Execution of the Security Penetration Test	4
3.1 Subject Under Test	4
3.2 Scope	4
3.3 Methodology	4
3.4 Events	5
4 Risk Assessment	6
4.1 CVSS	6
4.2 Risk Categories	7
4.3 Vulnerability States	8
5 Summary of Findings	9
6 Findings and Remediation	10
6.1 Finding	10
6.1.1 JWT None Algorithm Signature Bypass	10
6.2 Finding	10
6.2.1 User Enumeration via Security Question Endpoint	10
6.3 Finding	11
6.3.1 JWT Token Exposes Password Hash and Sensitive User Data	11
6.4 Finding	12
6.4.1 Sensitive Files Exposed via FTP Directory Listing	12
6.5 Finding	13
6.5.1 2FA TOTP Secret Exposed via Status Endpoint	13
6.6 Finding	14
6.6.1 Security Questions Publicly Accessible Without Authentication	14
6.7 Finding	15
6.7.1 Application Configuration Exposed via Admin Endpoint	15
6.8 Finding	16
6.8.1 SQL Injection - Error-Based Information Disclosure	16
6.9 Finding	17
6.9.1 IDOR - User Enumeration via /api/Users/ Endpoint	17

6.10 Finding	18
6.10.1 IDOR - Unauthorized Access to Individual User Records	18
6.11 Finding	19
6.11.1 IDOR - Unauthorized Access to User Shopping Baskets	19
6.12 Finding	20
6.12.1 IDOR - Complete Basket Items Exposure	20
6.13 Finding	21
6.13.1 SQL Injection - Authentication Bypass	21
7 Appendix A - Lists, Scripts and Raw Data	23
8 Appendix B - List of Abbreviations	24

DRAFT

Scope

OpenHack Security Agent was engaged by Bjoern Kimminich to conduct an external IT security investigation. The subject of the investigation was the “OWASP Juice Shop v19.1.1”.

The test was performed using a local instance of the application at `localhost:3333` in a dedicated test environment..

Objective

The objective of the IT security investigation was to identify vulnerabilities and security gaps which, in the event of misuse, would have an impact on the availability, integrity and confidentiality of the defined applications and/or systems and the data processed on them. The result of this project is a report that contains a management summary of all relevant findings and a compilation of recommended measures.

The technical IT security audit is not a substantive audit effort to ensure that all vulnerabilities and security gaps existing at the time of the audit are identified. There is a possibility that established safeguards will effectively prevent identification and exploitation of vulnerabilities and security gaps. The client acknowledges that this is not an indicator of deficiencies in engagement performance and that additional unidentified vulnerabilities and security gaps may exist.

Disclaimer

‘OpenHack Security Agent’ conducted the security penetration test on behalf of ‘Bjoern Kimminich’. This report has been prepared exclusively for ‘Bjoern Kimminich’. It is intended exclusively for internal use and is accordingly not designed to serve third parties as a basis for their decisions, unless agreed otherwise in writing. Third parties may not derive any rights or otherwise benefit from this engagement unless agreed otherwise in writing. Affiliated companies of the client are also considered “third parties” within the meaning of this engagement.

‘OpenHack Security Agent’ does not assume any liability or legal claims against third parties unless agreed otherwise in writing or a disclaimer cannot effectively be implemented.

It is the sole responsibility of anyone who becomes aware of the work results summarized in this report to decide to what extent and in what way the information is useful or appropriate and to supplement, check or update it as part of their own review.

No adjustments will be made to the report to reflect events or circumstances that occur after the report is issued, unless required by law.

The recommendations in this report are general and applicable in many different environments but could have unpredictable effects in specific environments. Therefore, any actions derived from the recommendations should be carefully considered and implemented through the regular change process. This should include appropriate testing of the changes on a test environment prior to going live. If the changes are not tested in advance in an identically configured test environment, failures or other damage cannot be ruled out.

Please note: Technical descriptions (or parts thereof) in this report may be taken from the OWASP Testing Guide (www.owasp.org).

1 Document Control

Document Status

Title	Security Assessment Report
Document Owner	OpenHack Security Agent
Classification	Strictly Confidential
Project Timeframe	2026-02-26

Version History

Version	Date	State	Author	Comments
0.1	2026-02-26	Draft	Jane Doe	-
1.0	2026-02-26	Final document	Jane Doe	-

Contact Persons - Assessor

Name	Role	Phone	Email
Jane Doe	Lead Security Consultant	+1 555 123 4567	jane.doe@openhack.sec
John Smith	Security Consultant	+1 555 234 5678	john.smith@openhack.sec

Contact Persons - Client

Name	Role	Phone	Email
Bjoern Kimminich	Project Lead	+1 555 345 6789	bjoern.kimminich@owasp.org

2 Executive Summary

OpenHack Security Agent (hereinafter referred to as “ASSESSOR”) has been commissioned by Bjoern Kimminich (hereinafter referred to as “CLIENT”) to carry out a penetration test.

The penetration test of the OWASP Juice Shop v19.1.1 application took place on **2026-02-26**.

For this purpose, the web application, accessible at <http://juiceshop:3000>, was tested from the perspective of an external attacker.

As part of the test, a total of 13 vulnerabilities were identified: 3 critical, 5 high, 5 medium, 0 low, and 0 informational.

Critical: **3** | High: **5** | Medium: **5** | Low: **0** | Informational: **0** | Total: **13**

Table 2.1: Overview of Findings

Severity Count	
Critical	3
High	5
Medium	5
Low	0
Informational	0
Total	13

A description of the scope and test environment can be found in Chapter 3. Chapter 4 presents the risk assessment methodology. Chapter 5 provides a summary of findings. Chapter 6 contains the detailed technical descriptions of the findings including remediation measures.

It is recommended that the remediation of the critical risk vulnerabilities be treated with the highest priority and countermeasures implemented as soon as possible. The vulnerabilities with high and medium risk should also be remedied in the short term. The low-risk vulnerabilities should be treated with lower priority due to their reduced risk, but should still be addressed.

3 Execution of the Security Penetration Test

3.1 Subject Under Test

3.2 Scope

The scope for the assessment was the following targets:

3.3 Methodology

The security penetration test of the OWASP Juice Shop v19.1.1 took place on 2026-02-26.

The web application was tested for security vulnerabilities across the following categories. This can be considered a guideline as penetration tests are a creative process and therefore the tests are not limited to what is given here. The base of these categories is the OWASP Top 10 for Web, which is fully covered by this penetration test approach.

Category	Description
Broken Access Control	Users can act outside their permissions, leading to unauthorized viewing, modification, or deletion of data.
Security Misconfiguration	Systems or cloud services are insecurely configured, e.g., through default passwords or unnecessarily enabled features.
Software Supply Chain Failures	Vulnerabilities arise from compromised third-party code, libraries, or build pipelines.
Cryptographic Failures	Sensitive data is exposed through missing, weak, or incorrectly implemented encryption.
Injection	Attackers inject malicious commands (e.g., SQL or shell code) through input fields that the system erroneously executes.
Insecure Design	Fundamental architectural flaws that exist before implementation make an application inherently insecure.
Authentication Failures	Flaws in identity verification allow attackers to take over sessions or guess passwords (brute force).

Category	Description
Software or Data Integrity Failures	Lack of verification of code or data sources leads to acceptance of untrusted updates or serialization data.
Security Logging & Alerting Failures	Attacks go undetected or responses are delayed due to missing logs or absent alert notifications.
Mishandling of Exceptional Conditions	Programs respond inadequately to unforeseen situations, which can lead to crashes or logic errors.

3.4 Events

During the test, the following restrictions or events occurred that may have affected the scope or depth of the assessment:

DRAFT

4 Risk Assessment

4.1 CVSS

The risk assessment of the vulnerabilities found is performed using the industry standard Common Vulnerability Scoring System v3.1 (hereinafter referred to as “CVSS”). This is a standard that can be used to uniformly assess the vulnerability of computer systems and the severity of security vulnerabilities. This makes it possible to compare vulnerabilities more effectively.

The Common Vulnerability Scoring System has a metric structure and is based on values that are divided into three groups: “Base”, “Temporal”, and “Environmental”. To determine the vulnerability scores, each group has its own rules. The values of the Base group are invariant in time and remain the same in different environments. In the Temporal group, the time dependency of a vulnerability is taken into account. For example, the vulnerability of a system to a particular vulnerability decreases over time as more and more countermeasures such as patches become known and available. The Environmental group incorporates various criteria of the specific IT environments into the vulnerability rating.

The CVSS ratings are numerical values on a scale of 0.0 to 10.0, with the highest vulnerability of a system occurring at a value of 10.0. Our rating is based on the Base Metrics (Base Score) and is composed of:

- Attack Vector (AV)
- Attack Complexity (AC)
- Privileges Required (PR)
- User Interaction (UI)
- Scope (S)
- Confidentiality (C)
- Integrity (I)
- Availability (A)

As penetration testers, we can only assess the general threats posed by a vulnerability through Base Metrics. However, we have no insight into the internal structures of our clients. Therefore, the assessment of the specific risks for the client’s systems and business processes must be done by the client. For this purpose, CVSS offers Environmental Metrics. This makes it possible to subsequently adjust the CVSS score of vulnerabilities after the test result has been

submitted before they are remedied. This changes the assessment of the risk and thus the urgency of remediation of a vulnerability. For more information, see [CVSS v3.1 Specification](#).

4.2 Risk Categories

The risk categories used in this report reflect the recommended priority for addressing the vulnerabilities identified during the penetration test. The categorization is based on the probability of exploitation and the significance of the impact. The risk value is calculated using the CVSS v3.1 standard:

Assessment	Risk Category Description
Critical	Critical vulnerabilities that allow an attacker to gain full access to the object under test and its sensitive information. It is possible to cause enormous damage to the client's reputation. Furthermore, these vulnerabilities may allow attackers to gain wide-ranging privileges, including to other systems or applications of the client that have not been investigated in detail within the scope of this project. Exploitation of the vulnerabilities is not prevented and can be reproduced with medium to low effort.
High	Vulnerabilities that may allow an attacker to manipulate sensitive data, damage the client's reputation, gain unauthorized access to sensitive information, or gain unauthorized access to applications or the client's infrastructure. The vulnerabilities are reproducible with medium to high effort.
Medium	Vulnerabilities that may allow an attacker to harm the client's reputation or gain unauthorized access to client data. The privileges that an attacker can gain by exploiting the vulnerabilities are limited.
Low	Security vulnerabilities that do not pose a threat in themselves, but which, for example, provide attackers with useful information about the network and systems. These vulnerabilities allow an attacker only very limited access.
Informational	Anomalies such as functional limitations or inconsistencies. These do not currently represent a risk and have no negative impact on the test object. Accordingly, no action is required. Nevertheless, countermeasures can be helpful for the functional and safety improvement of the test object. If necessary, this may give rise to risks under other circumstances.

4.3 Vulnerability States

The vulnerabilities can be in one of the following states:

- **Active:** The vulnerability has been identified and it has been possible to verify its existence through exploitation or direct evidence.
- **Potential:** The vulnerability has been identified but its exploitation has not been possible, so its existence cannot be fully verified, and it is up to the client to determine the impact.
- **Fixed:** The vulnerability has been remediated and verified through retesting.

DRAFT

5 Summary of Findings

Id	Finding Name	Risk Assessment	CVSS v3.1 Score
01	JWT None Algorithm Signature Bypass	Critical	10
02	User Enumeration via Security Question Endpoint	Medium	5.3
03	JWT Token Exposes Password Hash and Sensitive User Data	Critical	9.1
04	Sensitive Files Exposed via FTP Directory Listing	High	7.5
05	2FA TOTP Secret Exposed via Status Endpoint	Medium	6.5
06	Security Questions Publicly Accessible Without Authentication	Medium	6.5
07	Application Configuration Exposed via Admin Endpoint	Medium	5.3
08	SQL Injection - Error-Based Information Disclosure	Medium	5.3
09	IDOR - User Enumeration via /api/Users/ Endpoint	High	6.5
10	IDOR - Unauthorized Access to Individual User Records	High	6.5
11	IDOR - Unauthorized Access to User Shopping Baskets	High	6.5
12	IDOR - Complete Basket Items Exposure	High	6.5
13	SQL Injection - Authentication Bypass	Critical	10

6 Findings and Remediation

6.1 Finding

6.1.1 JWT None Algorithm Signature Bypass

Severity	Critical
CVSS Base Score	10.0 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H)
Assets	http://juiceshop:3000/api/Users/, http://juiceshop:3000/rest/user/login, All authenticated endpoints
Status	open

Description

The JWT authentication implementation accepts tokens with the 'none' algorithm, completely bypassing signature verification. An unauthenticated attacker can forge arbitrary JWT tokens with any claims (including admin role) without knowing the secret key. The forged tokens are accepted by all authenticated endpoints, enabling complete impersonation of any user including administrators.

Proof of Concept

Not provided

Remediation

Not provided

Evidence

- JWT None Algorithm Attack - Evidence showing forged JWT with alg:none accepted by server granting admin access: [images/finding-fb837520-5dd5-4976-902b-829179fb308c-2-jwt-none-algorithm-1772078933.txt](#)

6.2 Finding

6.2.1 User Enumeration via Security Question Endpoint

Severity	Medium
CVSS Base Score	5.3 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N)
Assets	http://juiceshop:3000/rest/user/security-question, GET /rest/user/security-question?email=
Status	open

Description

The security question endpoint GET /rest/user/security-question?email= allows unauthenticated enumeration of valid user accounts. When a valid email is provided, the endpoint returns the user's security question. When an invalid email is provided, it returns an empty object {}. This behavioral difference allows attackers to confirm the existence of user accounts, enabling targeted phishing or brute force attacks.

Proof of Concept

Not provided

Remediation

Not provided

Evidence

- User Enumeration - Evidence showing different responses for valid vs invalid email addresses: [images/finding-17ca2583-01c8-44d6-8702-b16b5542d549-3-user-enumeration-1772078860.txt](#)

6.3 Finding

6.3.1 JWT Token Exposes Password Hash and Sensitive User Data

Severity	Critical
CVSS Base Score	9.1 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N)
Assets	http://juiceshop:3000/rest/user/login, http://juiceshop:3000
Status	open

Description

The JWT authentication token returned upon login contains sensitive user data in its payload, including the user's password hash (MD5). This is a critical information disclosure vulnerability.

Vulnerable Endpoint: POST /rest/user/login

Exposed Data in JWT Payload: - Password hash (MD5): 0192023a7bbd73250516f069df18b500 for admin user - User role: admin - User email: admin@juice-shop - User ID: 1

Reproduction Steps: 1. Send POST request to /rest/user/login with valid credentials 2. Decode the returned JWT token (base64) 3. Observe the password hash and other sensitive data in the payload

Impact: - Password hashes can be cracked offline using rainbow tables or brute force (MD5 is weak) - Enables account takeover if password is cracked - Exposes user role allowing privilege escalation planning - Violates security best practices for JWT implementation

Evidence: JWT token payload decoded showing password hash: 0192023a7bbd73250516f069df18b500

Proof of Concept

Not provided

Remediation

Not provided

Evidence

- JWT token decoded payload showing password hash 0192023a7bbd73250516f069df18b500 exposed in token: [images/finding-68c4cdf-c629-4cd5-bc2c-753c3deeb1f8-8-jwt-token-decoded.json](#)

6.4 Finding

6.4.1 Sensitive Files Exposed via FTP Directory Listing

Severity	High
CVSS Base Score	7.5 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N)
Assets	http://juiceshop:3000/ftp/ , http://juiceshop:3000/ftp/incident-support.kdbx , http://juiceshop:3000/ftp/acquisitions.md
Status	open

Description

The /ftp/ endpoint exposes a directory listing containing sensitive files without authentication. Files include a KeePass password database, confidential business documents, and other sensitive data.

Vulnerable Endpoint: GET /ftp/

Exposed Sensitive Files: 1. incident-support.kdbx - KeePass password database (3246 bytes, confirmed KDBX format) 2. acquisitions.md - Confidential business acquisition plans 3. announcement_encrypted.md - Encrypted announcements 4. coupons_2013.md.bak - Backup coupon file 5. package.json.bak - Package configuration backup 6. suspicious_errors.yml - Error configuration 7. Multiple order PDF files with order details

Reproduction Steps: 1. Navigate to http://juiceshop:3000/ftp/ 2. Observe full directory listing without authentication 3. Download sensitive files (e.g., incident-support.kdbx) 4. Verify files contain sensitive data (KeePass database confirmed via file command)

Impact: - KeePass database could contain passwords for other systems - Confidential business documents exposed - Order PDFs may contain customer PII - Enables lateral movement if passwords are extracted

Evidence: Downloaded and verified KeePass database (KDBX format) and confidential acquisitions document

Proof of Concept

Not provided

Remediation

Not provided

Evidence

- KeePass password database (incident-support.kdbx) downloaded from FTP without authentication: [images/finding-3c2793e0-8416-499b-8716-6e6e58304c1e-9-ftp-incident-support.kdbx](#)
- Confidential acquisitions document downloaded from FTP directory: [images/finding-3c2793e0-8416-499b-8716-6e6e58304c1e-10-ftp-acquisitions.md](#)
- HTTP response showing FTP directory listing with sensitive files: [images/finding-3c2793e0-8416-499b-8716-6e6e58304c1e-11-http-ftp-directory-response.txt](#)

6.5 Finding

6.5.1 2FA TOTP Secret Exposed via Status Endpoint

Severity

Medium

CVSS Base Score

6.5 (CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N)

Assets	http://juiceshop:3000/rest/2fa/status
Status	open

Description

The 2FA status endpoint exposes the TOTP secret before 2FA is enabled, allowing an authenticated attacker to generate valid 2FA codes.

Vulnerable Endpoint: GET /rest/2fa/status

Exposed Data: - TOTP Secret: HVBTS LJAKU4BENJR - Setup Token: JWT containing the secret - User email

Reproduction Steps: 1. Authenticate as any user 2. Send GET request to /rest/2fa/status with Authorization header 3. Observe the TOTP secret in the response 4. Use the secret to generate valid TOTP codes

Impact: - Complete bypass of 2FA security control - Attacker can generate valid 2FA codes - Undermines multi-factor authentication - Enables persistent access to compromised accounts

Evidence: TOTP secret HVBTS LJAKU4BENJR exposed in API response

Proof of Concept

Not provided

Remediation

Not provided

Evidence

- 2FA status response exposing TOTP secret HVBTS LJAKU4BENJR: [images/finding-74f38d93-da28-46b8-84b5-19420f76135c-12-http-2fa-status-response.json](#)

6.6 Finding

6.6.1 Security Questions Publicly Accessible Without Authentication

Severity	Medium
CVSS Base Score	6.5 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:L/A:N)
Assets	http://juiceshop:3000/api/SecurityQuestions/
Status	open

Description

All security questions are publicly accessible without authentication, enabling targeted social engineering attacks.

Vulnerable Endpoint: GET /api/SecurityQuestions/

Exposed Data: All 14 security questions including: - “Your eldest siblings middle name?” - “Mother’s maiden name?” - “Mother’s birth date? (MM/DD/YY)” - “Father’s birth date? (MM/DD/YY)” - “What’s your favorite place to go hiking?” - etc.

Reproduction Steps: 1. Send GET request to /api/SecurityQuestions/ without any authentication 2. Observe full list of security questions returned

Impact: - Enables targeted social engineering attacks - Attackers can research answers to specific questions - Facilitates password reset attacks - Undermines security question mechanism

Evidence: Full list of 14 security questions accessible without authentication

Proof of Concept

Not provided

Remediation

Not provided

Evidence

- Security questions API response exposing all 14 questions without authentication: [images/finding-a27b6f60-5f9e-43cb-a3b6-4bd2181a5e10-13-http-security-questions-response.txt](#)

6.7 Finding

6.7.1 Application Configuration Exposed via Admin Endpoint

Severity	Medium
CVSS Base Score	5.3 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N)
Assets	http://juiceshop:3000/rest/admin/application-configuration
Status	open

Description

Full application configuration is exposed without authentication at the /rest/admin/application-configuration endpoint, revealing sensitive OAuth client IDs and internal settings.

Vulnerable Endpoint: GET /rest/admin/application-configuration

Exposed Data: - Google OAuth Client ID: 1005568560502-6hm16l ef8oh46hr2d98vf2ohlnj4nfhq . app
- Authorized OAuth redirect URIs - Internal server configuration (ports, base URLs) - Application settings and features - Product configurations - Chatbot training data reference

Reproduction Steps: 1. Send GET request to /rest/admin/application-configuration without authentication 2. Observe full application configuration returned in JSON

Impact: - OAuth client ID exposure enables OAuth-based attacks - Internal infrastructure details revealed - Attack surface mapping facilitated - May enable phishing with legitimate OAuth credentials

Evidence: Full JSON configuration including OAuth client ID exposed

Proof of Concept

Not provided

Remediation

Not provided

Evidence

- Application configuration response exposing OAuth client ID and internal settings: [images/finding-66e4e104-ec8e-44da-abbf-18c4da7a9eb9-14-http-app-config-response.txt](#)

6.8 Finding

6.8.1 SQL Injection - Error-Based Information Disclosure

Severity	Medium
CVSS Base Score	5.3 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N)
Assets	http://juiceshop:3000/rest/products/search
Status	open

Description

SQL injection vulnerability in the product search endpoint reveals database error messages. User-supplied input in the 'q' parameter triggers SQLite errors, exposing sensitive information about the technology stack.

Confirmed Impact: - Database type disclosed: SQLite - Technology stack exposed: Express.js
- Query structure information leaked via error messages - Enables further attack refinement

Reproduction Steps: 1. Send GET request to /rest/products/search?q=apple' 2. Observe SQLITE_ERROR: incomplete input in response 3. Error confirms SQL injection vulnerability

Root Cause: User-supplied input in the 'q' parameter is concatenated into SQL query without proper sanitization.

Note: While UNION-based extraction was not successful in initial testing, the error-based injection confirms vulnerability and enables further attack refinement.

Proof of Concept

Not provided

Remediation

Not provided

Evidence

- Error-based SQL injection evidence - SQLite error message disclosed: [images/finding-3b26cdbcd87f-424d-941e-d94b3c4829ef-7-sqli-search-endpoint-001.txt](#)

6.9 Finding

6.9.1 IDOR - User Enumeration via /api/Users/ Endpoint

Severity	High
CVSS Base Score	6.5 (CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N)
Assets	http://juiceshop:3000/api/Users/
Status	open

Description

The GET /api/Users/ endpoint allows any authenticated user to retrieve the complete list of all users in the system, including administrators. This is a severe Insecure Direct Object Reference (IDOR) vulnerability that exposes sensitive user data.

Vulnerable Endpoint: GET /api/Users/

Confirmed Impact: - All 39 users exposed including 7 administrators - User emails exposed (admin@juice-sh.op, ciso@juice-sh.op, etc.) - User roles disclosed (admin, deluxe, customer, accounting) - Deluxe tokens exposed for premium users - Profile image paths disclosed - Account creation timestamps exposed

Reproduction Steps: 1. Authenticate as any user (even low-privilege customer) 2. Send GET request to /api/Users/ with Authorization header 3. Receive complete list of all users with sensitive data

Root Cause: The `/api/Users/` endpoint lacks proper authorization checks to restrict access to administrators only. Any authenticated user can enumerate all accounts.

Evidence: `idor-all-users-response.json` shows complete user list with admin accounts exposed

Proof of Concept

Not provided

Remediation

Not provided

Evidence

- Complete user list exposed via IDOR - shows all 39+ users including admins with emails and roles: `images/finding-34528710-90d9-4378-9caa-bd1f63d3b60c-15-idor-all-users-response.json`

6.10 Finding

6.10.1 IDOR - Unauthorized Access to Individual User Records

Severity	High
CVSS Base Score	6.5 (CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N)
Assets	<code>http://juiceshop:3000/api/Users/1</code> , <code>http://juiceshop:3000/api/Users/{id}</code>
Status	open

Description

The GET `/api/Users/{id}` endpoint allows any authenticated user to access details of any other user by manipulating the user ID parameter. This is a classic Insecure Direct Object Reference (IDOR) vulnerability.

Vulnerable Endpoint: GET `/api/Users/{id}`

Confirmed Impact: - Accessed admin user details (ID 1) while authenticated as regular user (ID 39) - Retrieved admin email: `admin@juice-sh.op` - Confirmed admin role disclosure - Can access any of the 39+ user accounts by incrementing ID

Reproduction Steps: 1. Authenticate as a regular customer user (ID 39) 2. Send GET request to `/api/Users/1` with Authorization header 3. Receive full details of admin user account 4. Repeat with other user IDs (2, 3, 4, etc.) to access other users' data

Root Cause: The endpoint does not verify that the authenticated user has permission to access the requested user record. User IDs are sequential integers that can be easily enumerated.

Evidence: idor-admin-user-response.json shows admin user details accessed by non-admin user

Proof of Concept

Not provided

Remediation

Not provided

Evidence

- Admin user (ID 1) details accessed by non-admin user via IDOR - demonstrates unauthorized access to sensitive user data: [images/finding-66114857-dfca-4eeb-8d2b-224c5162c97b-16-idor-admin-user-response.json](#)

6.11 Finding

6.11.1 IDOR - Unauthorized Access to User Shopping Baskets

Severity	High
CVSS Base Score	6.5 (CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N)
Assets	http://juiceshop:3000/rest/basket/1 , http://juiceshop:3000/rest/basket/{id}
Status	open

Description

The GET /rest/basket/{id} endpoint allows any authenticated user to access the shopping basket of any other user by manipulating the basket ID parameter. This is an Insecure Direct Object Reference (IDOR) vulnerability that exposes user purchase intentions.

Vulnerable Endpoint: GET /rest/basket/{id}

Confirmed Impact: - Accessed admin user's basket (ID 1) while authenticated as regular user (ID 39) - Retrieved basket contents including products and quantities - Can enumerate all baskets to spy on other users' shopping activities - Exposes purchase intentions and shopping patterns

Reproduction Steps: 1. Authenticate as a regular customer user 2. Send GET request to /rest/basket/1 with Authorization header 3. Receive basket details for user ID 1 (admin) 4. Increment basket ID to access other users' baskets

Root Cause: The basket endpoint does not verify that the authenticated user owns the requested basket. Basket IDs are sequential integers that can be enumerated.

Evidence: idor-admin-basket-response.json shows admin basket accessed by non-admin user

Proof of Concept

Not provided

Remediation

Not provided

Evidence

- Admin user basket (ID 1) accessed by non-admin user via IDOR - shows basket structure and ownership: [images/finding-41bf7a00-7e98-4d61-8e56-ead804120a40-17-idor-admin-basket-response.json](#)

6.12 Finding

6.12.1 IDOR - Complete Basket Items Exposure

Severity	High
CVSS Base Score	6.5 (CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N)
Assets	http://juiceshop:3000/api/BasketItems/
Status	open

Description

The GET /api/BasketItems/ endpoint exposes ALL basket items from ALL users without any authorization filtering. This is an Insecure Direct Object Reference (IDOR) vulnerability that reveals shopping data across the entire user base.

Vulnerable Endpoint: GET /api/BasketItems/

Confirmed Impact: - All basket items from all users exposed in single response - Product IDs and quantities disclosed for all baskets - Basket IDs allow linking items to specific users - 7+ basket items from multiple users visible - Shopping patterns and preferences exposed

Reproduction Steps: 1. Authenticate as any user 2. Send GET request to /api/BasketItems/ with Authorization header 3. Receive complete list of all basket items across all users 4. Correlate BasketId with user baskets for full picture

Root Cause: The endpoint returns all basket items without filtering by the authenticated user's ownership. No authorization check restricts data to the current user's items only.

Evidence: idor-all-basket-items-response.json shows items from baskets 3, 4, 5, 8, 11 belonging to multiple users

Proof of Concept

Not provided

Remediation

Not provided

Evidence

- All basket items from all users exposed via IDOR - shows items from baskets 3,4,5,8,11 belonging to multiple users: [images/finding-0e13c8d6-1dcc-452e-9528-35dff261f4a9-18-idor-all-basket-items-response.json](#)

6.13 Finding

6.13.1 SQL Injection - Authentication Bypass

Severity	Critical
CVSS Base Score	10.0 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H)
Assets	http://juiceshop:3000/rest/user/login, http://juiceshop:3000/api/Users/, POST /rest/user/login
Status	open

Description

The login endpoint POST /rest/user/login is vulnerable to SQL injection in the email parameter. An unauthenticated attacker can bypass authentication entirely using SQL injection payloads.

Confirmed Exploitation: - Payload 1: ' OR 1=1-- - Returns valid JWT token for admin@juice-sh.op with admin role - Payload 2: admin' OR '1'='1'-- - Successfully bypasses authentication

Confirmed Impact: - Authentication bypass without valid credentials - Automatic login as admin user (admin@juice-sh.op) with full admin privileges - Password hash exposed in JWT

token payload - Complete user database extraction possible (39 users including 7 admins) - Deluxe tokens for premium users exposed

Reproduction Steps: 1. Send POST request to /rest/user/login with Content-Type: application/json 2. Use payload: {"email":"' OR 1=1--","password":"x"} 3. Receive valid JWT token for admin account 4. Use token to access protected resources like /api/Users/

Root Cause: User-supplied input in the email field is concatenated directly into SQL query without proper sanitization or parameterized queries.

Evidence: Multiple SQL injection payloads successfully bypassed authentication, returning admin JWT tokens.

Proof of Concept

Not provided

Remediation

Not provided

Evidence

- SQL Injection Authentication Bypass - Request/Response evidence showing admin JWT obtained via ' OR 1=1- payload: images/finding-52589287-93c7-4274-8145-35829fa76eb7-1-sqli-auth-bypass-001.txt
- Admin API Access via Stolen JWT - Evidence showing full user database (39 users) exposed using JWT from SQL injection: images/finding-52589287-93c7-4274-8145-35829fa76eb7-4-admin-api-access-1772078895.txt
- SQL injection authentication bypass - HTTP request/response showing admin token obtained: images/finding-52589287-93c7-4274-8145-35829fa76eb7-5-sqli-auth-bypass-001.txt
- Complete user database extracted using admin token from SQL injection bypass - 39 users including 7 admins: images/finding-52589287-93c7-4274-8145-35829fa76eb7-6-sqli-impact-user-data-001.json

7 Appendix A - Lists, Scripts and Raw Data

DRAFT

8 Appendix B - List of Abbreviations

Abbreviation	Description
API	Application Programming Interface
CAPTCHA	Completely Automated Public Turing test to tell Computers and Humans Apart
CVSS	Common Vulnerability Scoring System
FTP	File Transfer Protocol
IDOR	Insecure Direct Object Reference
MD5	Message-Digest Algorithm 5
OAuth	Open Authorization
ORM	Object-Relational Mapping
OWASP	Open Web Application Security Project
PII	Personally Identifiable Information
SQL	Structured Query Language
TOTP	Time-based One-Time Password
URI	Uniform Resource Identifier
WAF	Web Application Firewall

+-----+-----+